Documento Explicativo — Inciso A: Reconocimiento de Dígitos Manuscritos con Adaline (NumPy Puro)

Curso / Materia: Redes Neuronales / Internet de las Cosas
Estudiante / Autor: Práctica Académica Reproducible
Entorno de Ejecución: Python 3.10 / NumPy 2.2 / Matplotlib 3.10
Semilla Aleatoria Global: 42

---

1. Ficha Técnica y Resumen Ejecutivo

Parámetro / Métrica	Valor Observado / Configuración
Modelo	Adaline (*ADAptive LInear NEuron*) Multiclase (*One-vs-All*)
Frameworks de ML utilizados	**Ninguno** (Implementación artesanal con NumPy)
Conjunto de Datos	`NumerosManuscritos.csv` (5,000 imágenes de 20×20 píxeles)
Dimensiones de Entrada	$X \in \mathbb{R}^{5000 \times 400}$; con sesgo $X_b \in \mathbb{R}^{5000 \times 401}$
Matriz de Pesos Sinápticos	$W \in \mathbb{R}^{401 \times 10}$ (10 neuronas lineales en paralelo)
Partición Train / Test	80% entrenamiento (4,000 muestras) / 20% prueba (1,000 muestras)
Estrategia de Optimización	Descenso de Gradiente por Mini-Lotes (*Mini-Batch GD*), $B = 32$
Tasa de Aprendizaje (η)	0.01
Épocas de Entrenamiento	50
Error Cuadrático Medio Inicial (Época 1)	0.1180
Error Cuadrático Medio Final (Época 50)	0.0741
Exactitud en Entrenamiento (Train Accuracy)	**88.22%**
Exactitud en Prueba (Test Accuracy)	**85.10%** (851 aciertos de 1,000 muestras)

---

2. Objetivo de la Práctica

El objetivo fundamental del Inciso A es comprender y reproducir las bases matemáticas del aprendizaje supervisado neuronal temprano mediante el algoritmo **Adaline** (*Widrow & Hoff, 1960*), prescindiendo por completo de cajas negras o frameworks de alto nivel como TensorFlow, PyTorch o scikit-learn.

Se busca que el estudiante:
1. Valide, transponge y normalice tensores de imágenes almacenados en disco plano.
2. Comprenda el impacto del ordenamiento de memoria (estilo Fortran / MATLAB frente a estilo C) en datos visuales.
3. Formule e implemente vectorialmente la regla de aprendizaje delta y el descenso por gradiente.
4. Integre un esquema multiclase Uno-contra-Todos (*One-vs-All*) con objetivos bipolares ${-1, +1}$.
5. Analice los límites de la separabilidad lineal en visión por computadora a través de campos receptivos y matrices de confusión.

---

3. Inspección y Validación del Dataset (`NumerosManuscritos.csv`)

3.1. Dimensiones y Transposición

El archivo `datos/NumerosManuscritos.csv` contiene originalmente 400 filas y $5,000$ columnas.
- Cada columna representa una imagen digitalizada de un dígito manuscrito.
- Cada fila representa un píxel específico dentro de una rejilla de 20×20.
- Para alinearse con la convención estándar del aprendizaje automático $(N \text{ muestras}, D \text{ características})$, se aplica la transposición:

```
matriz_cruda = np.loadtxt(ruta_csv, delimiter=",")  # Forma: (400, 5000)
X = matriz_cruda.T                                  # Forma: (5000, 400)
```

3.2. Integridad Numérica y Rango de Píxeles

La verificación analítica del archivo arrojó:
- **Valores nulos (`NaN`):** 0
- **Valores infinitos (`Inf`):** 0
- **Rango numérico:** Mínimo de -0.1320 y máximo de 1.1277.
- **Media global:** 0.1313; **Desviación estándar:** 0.3012.

> **Nota metodológica crítica:** Los datos ya se encuentran previamente normalizados y escalados (alrededor del rango $[0, 1]$ con leves oscilaciones debidas a filtros de interpolación). Por este motivo, **no debe**

> **aplicarse una división ciega entre 255**, ya que esto reduciría los valores a un orden de magnitud diminuto $(\sim 10^{-3})$, impidiendo el aprendizaje.

3.3. Justificación del Supuesto de Etiquetas

El archivo CSV carece de una columna explícita con las clases verdaderas. En el código base se adopta la hipótesis de que las $5,000$ imágenes están ordenadas de manera canónica en 10 bloques consecutivos de 500 muestras por dígito ($0, 1, \dots, 9$):

```
y = np.arange(5000) // 500
```

Para validar empíricamente esta afirmación y no asumirla a ciegas, se extrajeron muestras visuales en los centros de cada bloque (índices 250, 750, 1250, etc.). La inspección visual confirmó que el bloque 0 corresponde fielmente al dígito 0, el bloque 1 al dígito 1, y así sucesivamente hasta el bloque 9.

3.4. Reconstrucción Espacial: Orden Fortran (`order='F'`)

El conjunto de datos original proviene de un entorno MATLAB (como la célebre base de datos de dígitos del curso de Stanford / Andrew Ng). En MATLAB, las matrices se aplanan en la memoria por columnas (*column-major order*), a diferencia de C y NumPy por defecto (*row-major order*).

Por ende, para reconstruir la imagen de forma erguida y legible es imperativo emplear:

```
def a_imagen(vector):
    return vector.reshape(20, 20, order="F")
```

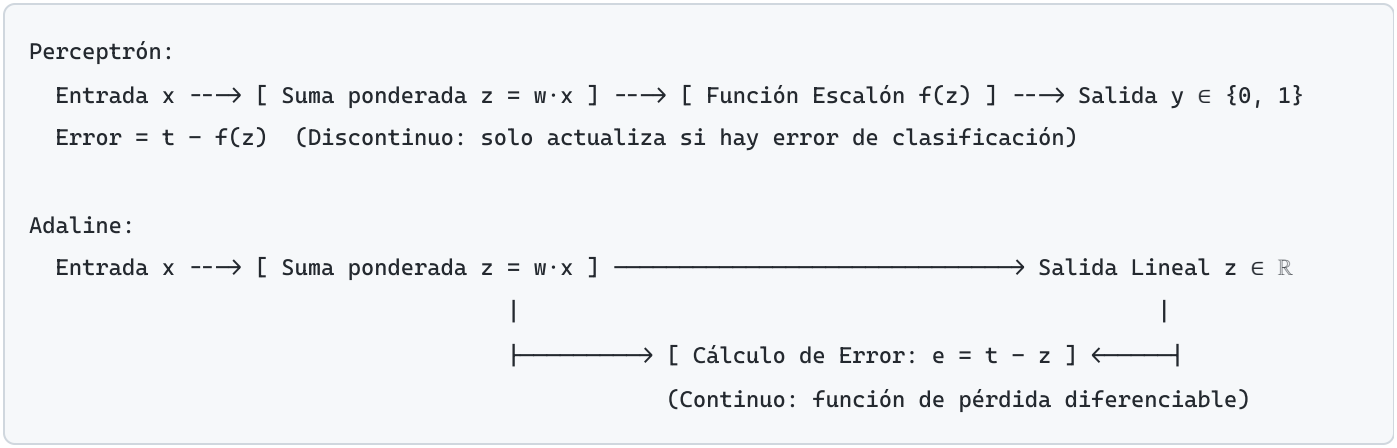
Si se empleara `order="C"`, el dígito aparecería reflejado diagonalmente y rotado 90 grados, afectando la coherencia espacial de las características.

---

4. Fundamento Teórico y Matemático de Adaline

4.1. Diferencia entre Perceptrón y Adaline

Aunque ambos fueron concebidos a finales de los años 50 y principios de los 60, existe una discrepancia matemática fundamental entre el Perceptrón de Frank Rosenblatt (1958) y Adaline de Bernard Widrow y Ted Hoff (1960):



1. **Superficie de Error Continua:** Adaline evalúa el error sobre la salida **lineal y continua** z, no sobre una salida discretizada. Esto define una superficie de error cuadrática, convexa y estrictamente diferenciable (un paraboloide), garantizando un único mínimo global.
2. **Capacidad de Convergencia:** Mientras que el Perceptrón oscila indefinidamente si las clases no son linealmente separables, Adaline minimiza el Error Cuadrático Medio por mínimos cuadrados ordinarios, encontrando la mejor frontera de compromiso lineal posible.

4.2. Función de Pérdida y Regla Delta (LMS / Widrow-Hoff)

La función de coste $J(W)$ para un mini-lote de tamaño B es el Error Cuadrático Medio (ECM):

$$J(W) = \frac{1}{2 B} \sum_{i=1}^B \sum_{k=1}^K \left(T_{i, k} - \hat{Y}_{i, k} \right)^2$$

donde $\hat{Y}_{i, k} = \mathbf{x}_i^T \mathbf{w}_k$ es la activación lineal de la neurona k ante la muestra i.

Calculando la derivada parcial con respecto a la matriz de pesos:

$$\nabla_W J = -\frac{1}{B} X_{\mathcal{B}}^T \left(T_{\mathcal{B}} - \hat{Y}_{\mathcal{B}} \right)$$

Aplicando la regla de actualización por gradiente descendente con tasa de aprendizaje η:

$$W \leftarrow W - \eta \nabla_W J = W + \frac{\eta}{B} X_{\mathcal{B}}^T \left(T_{\mathcal{B}} - X_{\mathcal{B}} W \right)$$

4.3. Estrategia Multiclase Uno-contra-Todos (*One-vs-All*)

Para extender Adaline (originalmente un clasificador binario) a un problema de 10 dígitos:
1. Se entrenan **10 neuronas Adaline de forma simultánea**, cada una representada por una columna de la matriz $W \in \mathbb{R}^{401 \times 10}$.
2. La matriz de objetivos para un lote de muestras se codifica con etiquetas bipolares:
$$T_{i, k} = \begin{cases} +1.0 & \text{si la muestra } i \text{ pertenece a la clase } k \\ -1.0 & \text{en caso contrario} \end{cases}$$
3. Durante la inferencia, no se aplica un umbral rígido independiente a cada neurona. En su lugar, se adopta la

regla de máxima activación (competencia lineal):
$$\hat{y}*i* = *arg**max*_k \right)$$} \left(\mathbf{x}_{b, i}^T \mathbf{w
La neurona que exhibe la mayor respuesta lineal reclama la predicción.

---

5. Preprocesamiento, Partición y Reproducibilidad

5.1. Vector de Sesgo (*Bias Trick*)

Para evitar gestionar el vector de sesgo w_0 como una variable independiente con su propia regla de actualización, se añade una columna de unos ($x_0 = 1$) a la izquierda de la matriz de características:

$$X_b = [\mathbf{1}_{N \times 1} \quad X] \in \mathbb{R}^{5000 \times 401}$$

De esta manera, el producto matricial $X_b W$ computa de forma compacta $\sum_{j=1}^{400} x_{ij} w_{jk} + w_{0k}$.

5.2. Partición 80/20 y Persistencia Compartida

Dado que el CSV original agrupa las muestras por dígito, una división no aleatoria provocaría un sesgo fatal. Se empleó una permutación con semilla fija 42:
- **Entrenamiento (80%):** 4,000 imágenes.
- **Prueba (20%):** 1,000 imágenes.

Para cumplir con el requerimiento de que el **Inciso B** utilice **exactamente la misma partición**, los índices se persistieron en `datos/particion_train_test.npz`:

```
np.savez_compressed("datos/particion_train_test.npz",
                    train_idx=train_idx,
                    test_idx=test_idx)
```

Distribución observada en prueba (1,000 muestras):
- Clase 0: 101 muestras
- Clase 1: 99 muestras
- Clase 2: 90 muestras
- Clase 3: 112 muestras
- Clase 4: 101 muestras
- Clase 5: 99 muestras
- Clase 6: 98 muestras
- Clase 7: 96 muestras
- Clase 8: 106 muestras

- Clase 9: 98 muestras
(Distribución prácticamente uniforme de ~ 100 imágenes por clase).

---

6. Explicación Detallada del Código de Entrenamiento

A continuación se detalla el bucle principal de entrenamiento implementado en `A_adaline_sin_frameworks.ipynb`:

```
# 1. Definición de hiperparámetros
TAM_LOTE = 32      # Tamaño de lote para Mini-Batch Gradient Descent
EPOCAS = 50        # Número de iteraciones completas sobre el set de train
LR = 0.01          # Tasa de aprendizaje (η)

# 2. Inicialización aleatoria gaussiana de pesos pequeños
# W tiene dimensión (401, 10): 401 entradas (400 píxeles + 1 sesgo) x 10 neuronas
W = rng.normal(0, 0.01, size=(n_pixeles + 1, N_CLASES))

historial_ecm, historial_acc = [], []

# 3. Bucle de épocas
for epoca in range(EPOCAS):
    # Barajar el orden de los datos de entrenamiento en cada época
    orden = rng.permutation(n_train)

    # Bucle de mini-lotes
    for ini in range(0, n_train, TAM_LOTE):
        lote = orden[ini:ini + TAM_LOTE]
        Xb, Tb = X_train_b[lote], T_train[lote]

        # Propagación hacia adelante: activación puramente lineal (sin función escalón)
        salida = Xb @ W                  # Dimensión: (|lote|, 10)

        # Cálculo de la discrepancia (objetivo deseado - obtenido)
        error = Tb - salida              # Dimensión: (|lote|, 10)

        # Gradiente medio sobre las muestras del lote
        grad = Xb.T @ error / len(lote)  # Dimensión: (401, 10)

        # Actualización de pesos según la Regla Delta
        W += LR * grad

    # 4. Cálculo de métricas al cierre de la época sobre todo el conjunto de entrenamiento
    salida_tr = X_train_b @ W
    ecm = np.mean((T_train - salida_tr) ** 2) / 2
    acc = np.mean(np.argmax(salida_tr, axis=1) == y_train)
    historial_ecm.append(ecm)
    historial_acc.append(acc)
```

7. Resultados y Curvas de Aprendizaje

El modelo exhibió un comportamiento convergente y asintótico a lo largo de las 50 épocas:

Época	Error Cuadrático Medio (ECM)	Exactitud Entrenamiento (%)
1	0.1180	80.42%
5	0.0910	85.08%
10	0.0842	86.88%
20	0.0781	88.05%
30	0.0759	88.02%
40	0.0749	87.90%
50	**0.0741**	**88.22%**

Interpretación de las Curvas (`paso4_entrenamiento.png`)

1. **Descenso suave del ECM:** No se observan divergencias ni oscilaciones descontroladas, lo cual demuestra que la tasa $\eta = 0.01$ combinada con el lote $B=32$ proporciona un equilibrio óptimo entre velocidad de convergencia y estabilidad numérica.
2. **Rendimiento de clasificación:** La exactitud se incrementa rápidamente durante las primeras 10 épocas (del 80.4% al 86.9%) y alcanza su meseta alrededor de la época 35, fluctuando en torno al 88.2%.

---

8. Evaluación en el Conjunto de Prueba e Interpretación de Gráficas

8.1. Métricas Globales en Test

Al evaluar el modelo sobre las 1,000 imágenes de prueba que no participaron en el entrenamiento:
- **Total de muestras:** 1,000
- **Aciertos:** 851
- **Errores:** 149
- **Exactitud en Prueba (Test Accuracy): 85.10%**
- **Brecha de generalización (Train - Test):** $88.22\% - 85.10\% = 3.12\%$.

> La baja brecha (3.12%) evidencia que el modelo lineal no sufre de sobreajuste (*overfitting*). Por el contrario, su error dominante es el sesgo inductivo (*underfitting* estructural) debido a la naturaleza estrictamente lineal de sus fronteras.

8.2. Análisis de Predicciones Muestra (`paso5_predicciones.png`)

En las 10 muestras aleatorias evaluadas:
- Se lograron 9 aciertos claros en dígitos 1, 3, 6, 7 y 8.
- Se observó un error característico: un dígito 7 escrito sin barra horizontal intermedia fue clasificado como 1. Para un clasificador lineal, la presencia de trazos casi exclusivamente verticales en el centro de la imagen genera una activación mayor para la plantilla del 1 que para la del 7.

8.3. Matriz de Confusión (`matriz_confusion_adaline.png`)

La matriz de confusión revela patrones de error altamente predecibles:
1. **Dígito 0 (97/101) y Dígito 6 (95/98):** Muy alta precisión. Sus siluetas cerradas y bucles inferiores son geométricamente disyuntos de otros caracteres.
2. **Dígito 1 (95/99):** Alta precisión debido a su trazo vertical concentrado.
3. **Dígito 5 (72/99):** La clase más castigada. Sufrió **11 confusiones con el dígito 8**. Ambos dígitos comparten la mitad inferior curvada y trazos horizontales superiores; un modelo lineal no puede discernir con nitidez si el lazo superior está cerrado (8) o abierto a la derecha (5).
4. **Dígito 2 (72/90):** Confundido frecuentemente con 8, 1 y 6.
5. **Dígito 9 (84/98):** Confundido 9 veces con el 7 debido al trazo diagonal compartido y la barra superior.

8.4. Visualización de los Campos Receptivos (`pesos_receptivos_adaline.png`)

Al visualizar las matrices de pesos de 20×20 correspondientes a cada neurona utilizando un mapa de color divergente (*seismic*):
- **Neurona 0:** Presenta un núcleo central azul intenso (inhibitorio, penaliza si hay tinta en el centro) rodeado por una corona elíptica roja (excitatoria, premia si hay tinta en el perímetro).
- **Neurona 1:** Muestra una columna central vertical roja delgada flanqueada por bandas azules a izquierda y derecha.
- **Neurona 7:** Resalta en rojo la barra superior horizontal y la diagonal descendente, con regiones inhibitorias en la esquina inferior izquierda.
- **Neurona 8:** Exhibe dos núcleos centrales inhibitorios correspondientes a los dos "huecos" del número ocho, y anillos excitatorios circundantes.

Esta visualización confirma de forma tangible que **cada neurona Adaline actúa como una plantilla de correlación espacial rígida**.

---

9. Conclusiones Académicas y Transición al Inciso B

1. **Eficacia del Modelo Lineal Simple:**
   A pesar de no contar con capas ocultas ni funciones de activación complejas, Adaline demostró que un conjunto de 10 neuronas lineales optimizadas con la regla delta puede clasificar dígitos centrados de 20×20 con más de un 85% de exactitud.

2. **Limitaciones Intrínsecas:**
   Adaline es ciego a la invarianza espacial frente a desplazamientos, rotaciones o curvaturas no alineadas. Al ser una combinación lineal ponderada de píxeles independientes, no puede computar relaciones de orden superior (por ejemplo, "si hay una curva superior Y una línea inferior, pero NO un trazo intermedio").
3. **Fundamentación para el Inciso B:**
   Para superar el techo del $\sim 85\%$ y resolver las ambigüedades entre los dígitos 5, 8, 7 y 9, es necesario introducir no-linealidades mediante funciones de activación como ReLU y arquitecturas profundas (MLP con capas ocultas). Este será el objeto de estudio en el **Inciso B**, empleando TensorFlow y Keras sobre esta misma partición de datos.